

Guía básica de Git y GitHub desde la terminal

Todo lo que un perfil UX necesita para versionar, respaldar y compartir su prototipo. Sin vueltas.

Setup inicial (una sola vez en tu vida)

```
# Configura tu nombre y email (aparecen en cada commit)
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"

# Instala GitHub CLI
#   Mac:    brew install gh
#   Win:    winget install GitHub.cli
#   Linux:  sudo apt install gh

# Conéctate a GitHub desde la terminal
gh auth login
```

El flujo básico de todos los días

Piensa en git como un “guardar partida” del proyecto: cada commit es un punto al que siempre puedes volver.

```
# Inicializar un repo nuevo (una vez por proyecto)
git init
git branch -M main

# Ciclo diario: cambias archivos y luego...
git status          # ver qué cambió
git add .           # preparar todos los cambios
git commit -m "mensaje claro" # guardar el snapshot

# Subir a GitHub la primera vez
gh repo create mi-proyecto --public --source=. --remote=origin
git push -u origin main

# Subir cambios posteriores
git push
```

Trabajo con ramas (branches)

Una rama es una copia paralela para experimentar sin romper lo que ya funciona.

```
git checkout -b feature/nueva-pantalla # crear y cambiarse a una rama
git checkout main                      # volver a main
git merge feature/nueva-pantalla       # traer la rama a main
git branch -d feature/nueva-pantalla   # borrar la rama (post-merge)
```

Convención de mensajes de commit

Prefijo	Cuándo usarlo
feat:	Nueva funcionalidad. Ej: feat(fertilidad): agregar tipo de límite ciclo
fix:	Corrección de bug. Ej: fix(tin): corregir badge vendor-check-pending
style:	Cambios visuales sin lógica. Ej: style(tabla): spacing según design.md
refactor:	Reestructurar sin cambio funcional.
docs:	Documentación (claude.md, design.md, README).
chore:	Mantenición: dependencias, configs.

Tip: Un buen commit responde “¿qué cambió y dónde?” en una línea. Tu yo del futuro (y tu dev) te lo van a agradecer.

Comandos de rescate (cuando algo se rompe)

```
git log --oneline      # ver historial de commits
git checkout <hash>    # visitar un commit anterior (solo lectura)
git reset --hard <hash> # VOLVER a un commit y BORRAR lo posterior
git revert <hash>      # crear un commit que deshace otro
git stash              # guardar cambios sin commitear y limpiar
git stash pop          # recuperar esos cambios guardados
```

Ojo: `git reset --hard` no tiene deshacer. Si dudas, usa `git stash` o `git revert`, que son reversibles.

Chuleta mental

El 90% de tu vida con git son estos 4: **git status** → **git add .** → **git commit -m "..."** → **git push**. El resto lo googleas cuando lo necesites.